Lab 7(A) - Laplacian Filter

In [1]:
```
import cv2
import numpy as np

from IPython.display import Image, display
from matplotlib import pyplot as plt

plt.rc('text', usetex=False)
```

Apply Laplacian Filter on Image to get Sharp Image

In [2]:
```
# ================ ( Laplacian Filter ) ===================


# 0. Image file path
imgFile = "moon.jpg"

# 1. Load image and convert to grayscale
img = cv2.imread(imgFile, 0)

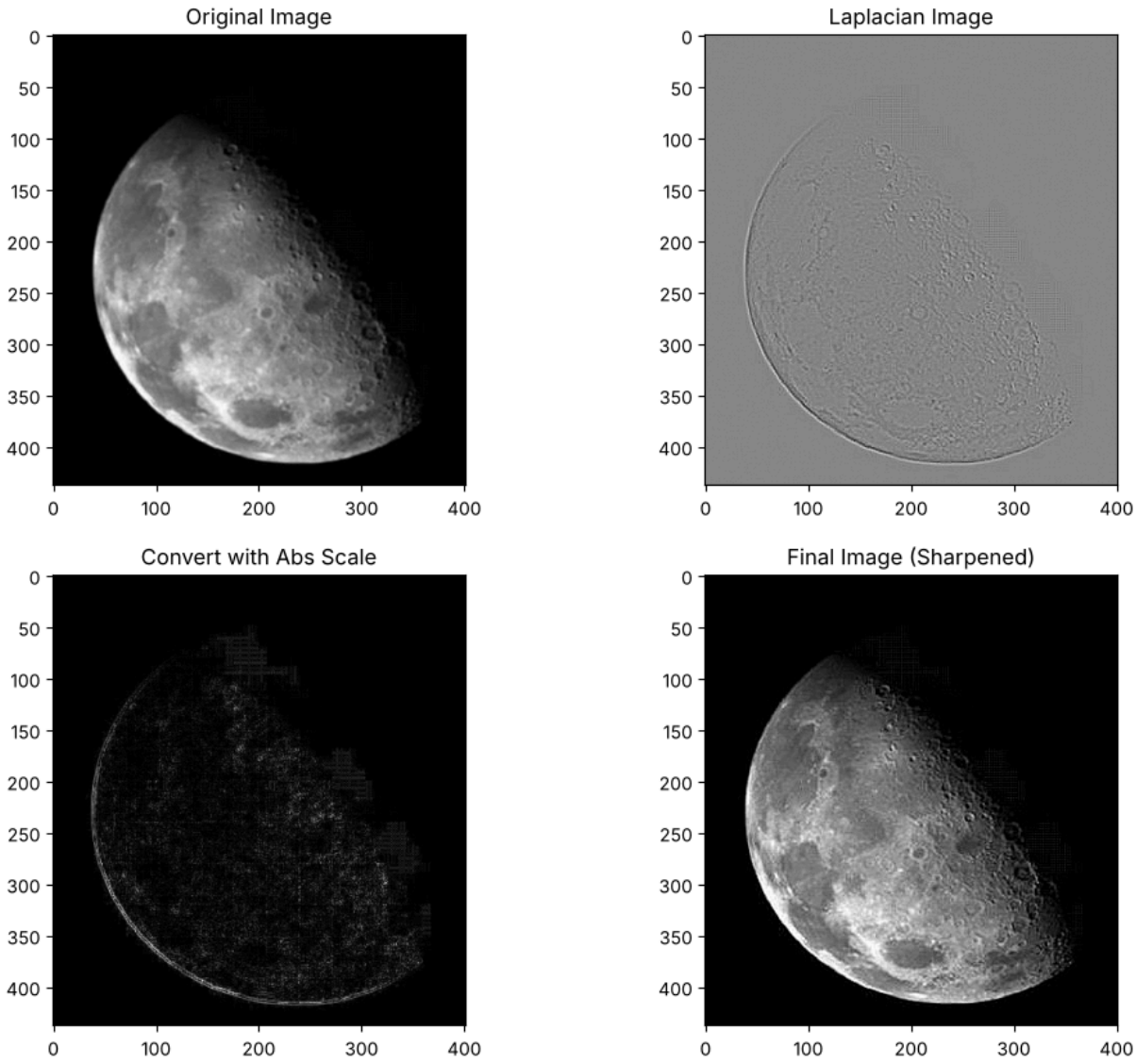
# 2. Manually define the Laplacian Kernel
# This is the standard 3x3 kernel that detects horizontal and vertical edges
manual_kernel = np.array([
    [0,  1,  0],
    [1, -4,  1],
    [0,  1,  0]
], dtype=np.float32)

# 3. Apply the filter using filter2D
# Use cv2.CV_64F to avoid clipping negative values
laplacian_manual = cv2.filter2D(img, cv2.CV_64F, manual_kernel)

# 4. Take absolute value and convert back to 8-bit (0-255)
laplacian_abs = cv2.convertScaleAbs(laplacian_manual)

# 5. Sharpen: g(x,y) = f(x,y) - Laplacian  (center coefficient is NEGATIVE, so we s
# Done in float and clipped, because uint8 "img + lap" wraps around above 255
finalImage = np.clip(img.astype(np.float64) - laplacian_manual, 0, 255).astype(np.u

# 6. Display images
plt.figure(figsize=(12, 10))
plt.subplot(221), plt.imshow(img, cmap='gray'), plt.title('Original Image')
plt.subplot(222), plt.imshow(laplacian_manual, cmap='gray'), plt.title('Laplacian I
plt.subplot(223), plt.imshow(laplacian_abs, cmap='gray'), plt.title('Convert with A
plt.subplot(224), plt.imshow(finalImage, cmap='gray', vmin=0, vmax=255), plt.title(
plt.show()
```



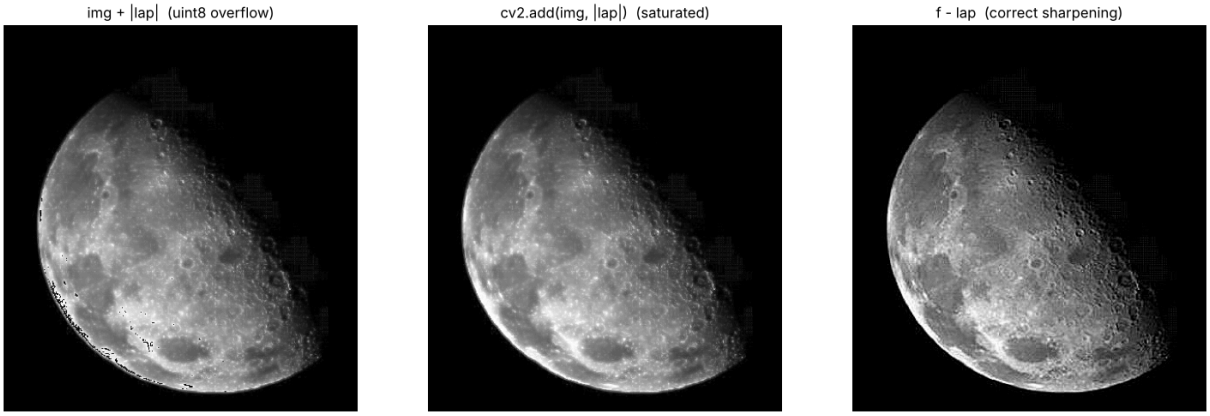
Why the original `img + laplacian_abs` is wrong

`img` and `laplacian_abs` are both `uint8`, so NumPy addition **wraps around** (e.g. 250 + 10 = 4), creating black speckles on bright areas. Also, since the kernel's center is **negative** (-4), the correct sharpening formula is $\mathbf{g = f - \nabla^2 f}$. The comparison below shows the difference.

In [3]:
```
wrong = img + laplacian_abs                    # uint8 overflow (wrap-around)
saturated = cv2.add(img, laplacian_abs)        # saturates at 255 instead of wrapp

plt.figure(figsize=(15, 5))
plt.subplot(131), plt.imshow(wrong, cmap='gray'), plt.title('img + |lap|  (uint8 ov
plt.subplot(132), plt.imshow(saturated, cmap='gray'), plt.title('cv2.add(img, |lap|
plt.subplot(133), plt.imshow(finalImage, cmap='gray', vmin=0, vmax=255), plt.title(
for ax in plt.gcf().axes: ax.axis('off')
plt.tight_layout()
plt.show()
```

```
print('Pixels that overflowed with img + |lap|:', int(np.sum(img.astype(int) + lapl
```



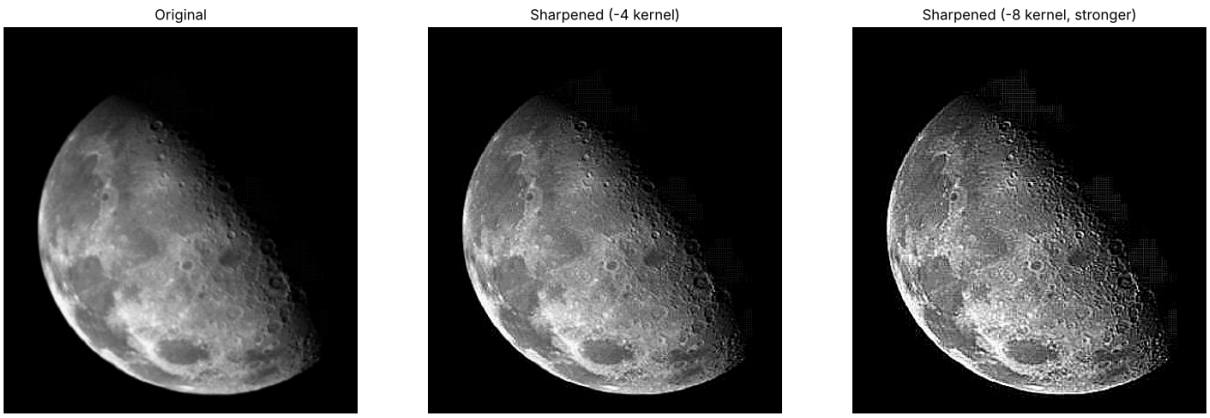

```
Pixels that overflowed with img + |lap|: 541
```

Alternative kernel (includes diagonal edges)

In [4]:
```
# Alternative kernel: Detects diagonal edges as well
diag_kernel = np.array([
    [1,  1,  1],
    [1, -8,  1],
    [1,  1,  1]
], dtype=np.float32)

laplacian_diag = cv2.filter2D(img, cv2.CV_64F, diag_kernel)
finalDiag = np.clip(img.astype(np.float64) - laplacian_diag, 0, 255).astype(np.uint

plt.figure(figsize=(15, 5))
plt.subplot(131), plt.imshow(img, cmap='gray'), plt.title('Original')
plt.subplot(132), plt.imshow(finalImage, cmap='gray', vmin=0, vmax=255), plt.title(
plt.subplot(133), plt.imshow(finalDiag, cmap='gray', vmin=0, vmax=255), plt.title('
for ax in plt.gcf().axes: ax.axis('off')
plt.tight_layout()
plt.show()
```



Built-in `cv2.Laplacian` with Gaussian pre-blur

In [5]:

```
# ================= ( Laplacian Filter ) ===============

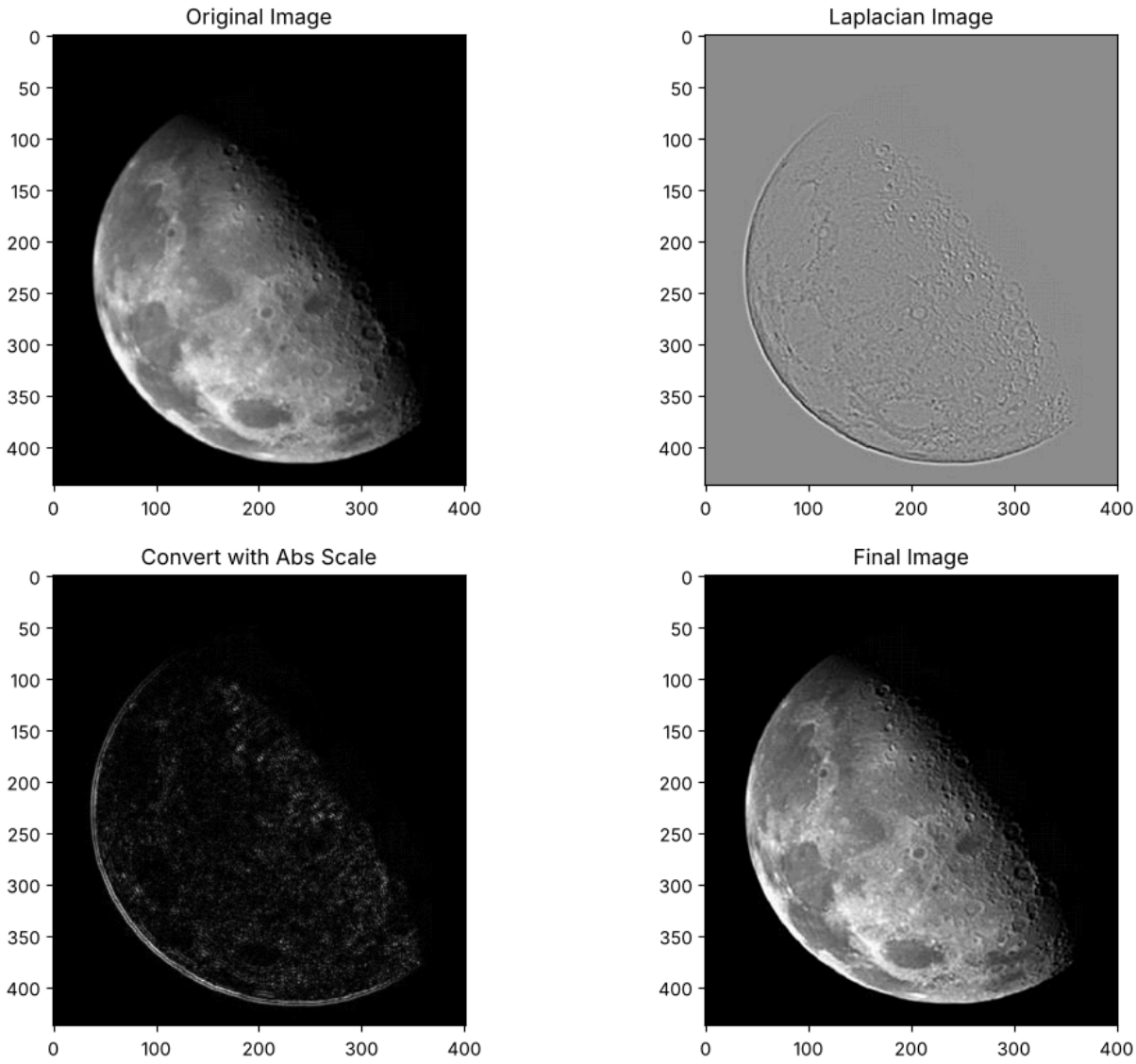
# 1. Optional: Apply Gaussian Blur to reduce noise
# This prevents the algorithm from picking up tiny details as edges
blurred = cv2.GaussianBlur(img, (3, 3), 0)

# 2. Apply the Laplacian Algorithm
# Use CV_64F to capture both positive and negative gradients (essential for edges)
laplacian = cv2.Laplacian(blurred, cv2.CV_64F)

# 3. Convert back to 8-bit image to view it properly
laplacian_8bit = cv2.convertScaleAbs(laplacian)

# 4. Subtract Laplacian from the original image (cv2.Laplacian uses the -4 center k
finalImage2 = np.clip(img.astype(np.float64) - laplacian, 0, 255).astype(np.uint8)

# 5. Display images
plt.figure(figsize=(12, 10))
plt.subplot(221), plt.imshow(img, cmap='gray'), plt.title('Original Image')
plt.subplot(222), plt.imshow(laplacian, cmap='gray'), plt.title('Laplacian Image')
plt.subplot(223), plt.imshow(laplacian_8bit, cmap='gray'), plt.title('Convert with
plt.subplot(224), plt.imshow(finalImage2, cmap='gray', vmin=0, vmax=255), plt.title
plt.show()
```

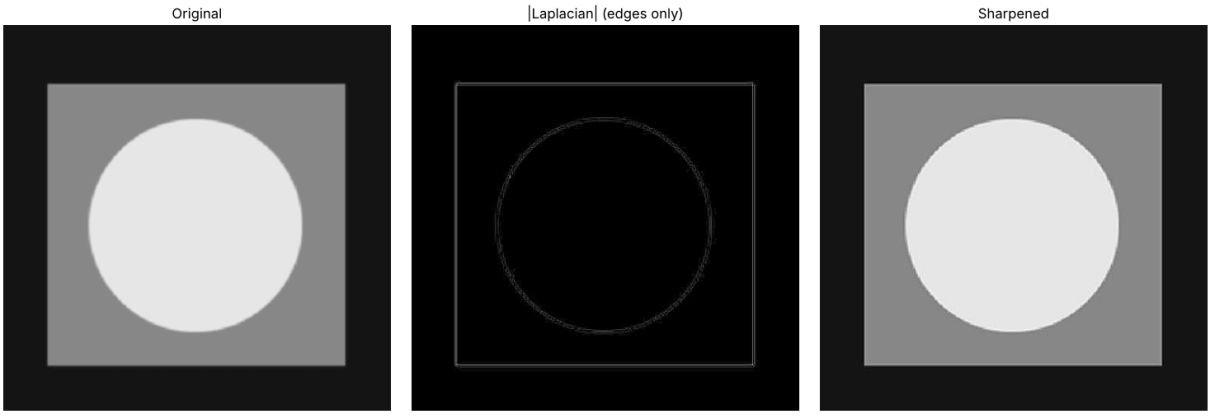


Laplacian on a simple shape (square + circle)

In [6]:
```
shape = cv2.imread('square.jpg', 0)

lap_shape = cv2.Laplacian(shape, cv2.CV_64F)
lap_shape_abs = cv2.convertScaleAbs(lap_shape)
sharp_shape = np.clip(shape.astype(np.float64) - lap_shape, 0, 255).astype(np.uint8

plt.figure(figsize=(15, 5))
plt.subplot(131), plt.imshow(shape, cmap='gray', vmin=0, vmax=255), plt.title('Orig
plt.subplot(132), plt.imshow(lap_shape_abs, cmap='gray'), plt.title('|Laplacian| (e
plt.subplot(133), plt.imshow(sharp_shape, cmap='gray', vmin=0, vmax=255), plt.title
for ax in plt.gcf().axes: ax.axis('off')
plt.tight_layout()
plt.show()
```



Observations

1. The Laplacian is a **second-derivative** filter: it is zero in flat regions and responds strongly at edges and fine details (crater rims on the moon, borders of the square and circle). Flat areas of the square/circle image become black in the |Laplacian| image.
2. The raw Laplacian contains **negative values**, so it must be computed in `CV_64F`; computing it in `uint8` would clip all negative responses to 0 and lose half the edge information.
3. Because the kernel's center is negative, sharpening is $\mathbf{f - \nabla^2 f}$. Adding instead of subtracting (and doing it in `uint8`) produces wrap-around artifacts.
4. The **−8 kernel** (with diagonals) responds more strongly and gives a more aggressive sharpening than the −4 kernel, but also amplifies noise more.
5. A small **Gaussian blur before the Laplacian** (LoG-like) suppresses noise so that only real edges are enhanced, giving a cleaner but slightly softer result.